Elevation

A Daily Shape-Stacking Minigame — Hackathon Planning & Design Doc

Prepared by Joshua Nguyen · Draft v3 · October 24, 2026

This document captures the full design and planning conversation for Elevation so it can be handed directly to an AI coding assistant (or a new teammate) with no missing context. Sections marked TBD are open decisions the team still needs to make — they are flagged, not blocking. Sections marked LOCKED reflect architecture decisions made in this revision and should be treated as settled unless something forces a change.

---

1. Concept Overview

Elevation is a 2D, browser-based daily puzzle game: players get a fixed set of shapes and two minutes to stack them as high and as stable as possible. After the build timer ends, the tower is tested against a natural disaster. Score comes from how tall the tower is and how long it survives the test.

1.1 Closest comparisons

- Tricky Towers — physics-based stacking and balance under pressure.
- Wordle — one puzzle per day, identical for every player, come back tomorrow for a new one.
- Krillion-style results screens — after finishing, you see your percentile against everyone else who played that day's puzzle.

1.2 Hackathon context

This is being built for a hackathon. Four tracks are in scope, and they should actively shape prioritization, not just be checked off at the end:

- Cursor + Grok Bot sponsor track — build using Cursor for agentic coding and Grok Bot for workflow orchestration across the repo.
- Best Novice — qualification is automatic (more than 50% first-time hackers on the team); no design implications.
- Best Pitch — the daily-puzzle hook and percentile leaderboard are the pitch's strongest material — lead with those.
- Most Entertaining — treat this as the primary design target. Under time pressure, prioritize satisfying collapse physics, disaster "drama," and juicy feedback (sound, screen shake, particle effects) over adding more shapes or disaster types.

2. Core Gameplay Loop

2.1 Game phases

- Onboarding — a short tutorial covering controls and the goal (always skippable).

- Building Phase — player gets all of the day's pieces, drawn from the current materials pool (Section 3), in a tray and 2 minutes to place them.
- Testing Phase — the finished tower is scored for height, then subjected to that day's disaster while a survival timer runs.
- Results — final height band, stability score, combined score, and leaderboard percentile.

2.2 Controls (desktop, primary target)

- Drag and drop to pick up a piece from the tray and move it anywhere on the canvas.
- Rotate with Q (counter-clockwise) and E (clockwise) while holding or just after placing a piece.
- Release to drop/place the piece at its current position and rotation.

TBD *— Mobile controls are a separate pass — rotation especially needs a touch-friendly equivalent (on-screen buttons vs. a twist gesture). Not required for the hackathon MVP since desktop is the priority.*

TBD *— Rotation increment per Q/E press isn't set yet. A reasonable default is 15° per press; confirm by feel during playtesting.*

2.3 Piece tray and placement rules

- All of the day's pieces are visible in the tray at once — the player chooses which one to place and in what order; nothing is hidden or queued.
- Pieces can be placed anywhere on the canvas, not just on top of the current stack.
- Already-placed pieces can be picked back up and moved. Doing so can destabilize whatever was leaning on it — that's an intended risk, not a bug.
- A piece that ends up away from the tower, or is never placed, simply disappears / isn't scored — no separate penalty beyond losing the use of that piece.

3. Materials System

3.1 Prototype material set

For the hackathon build, start with three basic shapes only:

Shape	Relative Mass	Relative Size	Friction
Square	Medium	Medium	Medium
Triangle	Low	Small–Medium	Medium
Pentagon	High	Medium–Large	Medium-High

TBD *— Exact numeric values for mass, size, and friction per shape are placeholders — tune these by playtesting the build feel.*

3.2 Material properties

- Mass and size drive weight distribution and footprint; both affect how easily a piece tips.
- Friction varies per material and affects how well pieces grip each other when stacked at an angle.

- No bounciness (restitution = 0) for any material — a falling piece should settle or topple, never bounce.
- Center of mass matters — it's the geometric centroid of the shape, not assumed uniform at a single point, so an off-center load tips a piece realistically.

3.3 Daily material assignment

- One global set per day — a random draw of 10 materials from the current materials pool (Section 3.1 during MVP, Section 3.4's library as it grows), generated once per day via a single global seed and identical for every player, like Wordle's daily word.
- One attempt per device per day using that day's set (Section 7).
- There is no separate per-item mechanism anymore — every piece in the tray, however it was authored, is drawn from the same pool by the same seed. See Section 6 for the full seed mechanics.

3.4 Target materials system — a library of random real-world objects

LOCKED — The intended end point for Elevation's materials system is not a fixed set of shapes at all — it's a large, growing library of literally any real-world object (a ketchup bottle, an elephant, a traffic cone, a couch), with physical properties estimated rather than hand-authored. The three basic shapes in Section 3.1 are explicitly a starting placeholder to validate the build feel, not the target.

This is built as an offline content-generation pipeline, not a runtime dependency — the game never calls SerpAPI or Gemini during play. Items are generated ahead of time (in batches, whenever the team runs the script) and saved into a static library that the daily seed then draws from exactly like it would draw from the basic-shape list.

Pipeline (run offline, in batches, independent of daily play)

- Start from a list of candidate object names — hand-brainstormed, or sourced by querying SerpAPI for themed categories ("funny household objects," "zoo animals," "kitchen items") and pulling names from the results.
- For each name, call SerpAPI's image search once to get a reference image.
- Call Gemini with the object name + reference image, using structured JSON output, asking it to estimate: archetype (wide-stable / tall-narrow / round / flat-wide / irregular), relative size, relative density, footprint ratio, friction, and restitution — see the worked example and schema from the architecture discussion earlier in this project.
- Convert the response into a full item definition — including a simple physics-shape approximation (box, circle, or basic convex polygon; never a literal silhouette hitbox) — and append it to the materials library (a static JSON/TS file committed to the repo, e.g. src/data/items.generated.json, or a Supabase items table if the team wants to keep growing it without redeploying).
- Visually, each item is rendered as a simple flat-icon silhouette in the game's existing minimalist style (Section 8), not as the raw SerpAPI photo — keeps the art direction consistent. The SerpAPI image is reference material for whoever draws or auto-generates that icon, not the in-game asset itself.
- Once an item is in the library, it behaves exactly like a basic shape to every other system — the daily seed, the physics engine, and the scoring system have no special case for "real object" vs. "basic shape."

How much of the library gets built during the hackathon itself is a capacity question, not a design question — the pipeline and schema are the same whether the library ends up with 5 items or 500. Section 10 covers how this fits the roadmap and who owns it.

TBD — *Size of the initial library isn't set — even a batch of 15–20 items is enough to replace the 3 basic shapes for a demo and prove the pipeline works end to end.*

TBD — *Whether the library lives as a committed static file or a Supabase table isn't decided — static file is simpler and is the hackathon-appropriate default; a table only helps if the team wants to keep adding items without a redeploy.*

3.5 Breaking and durability (stretch goal)

Planned direction: each piece has a health bar and visibly cracks under stress before breaking outright. A further idea — pieces that are well-supported by the surrounding structure become stronger — is unsolved.

TBD — *The breaking/health system and any structural-reinforcement bonus are explicitly out of MVP scope. Revisit after the core build-and-test loop works; don't let this block Phase 1–3 of the roadmap.*

4. Physics System

This is the technical core of the game. The sections below turn the rules from the design conversation into concrete, implementable mechanics.

4.1 Engine recommendation

LOCKED — Matter.js. It's 2D-only (matching this game exactly), runs directly on an HTML canvas, ships with built-in polygon bodies, and has a large base of drag-and-drop examples — which matters for hackathon speed. If the team later needs a more robust/deterministic solver (for example, for exact replay of the daily disaster), Planck.js (a Box2D port) is the fallback, but it's unnecessary complexity for the MVP.

4.2 Coordinate and scale system

Define a fixed constant, e.g. PX_PER_METER, and build all shape and platform dimensions in meters internally, converting to pixels only at render time. Matter.js's default tuning assumes roughly human-scale meter units, so working in meters first keeps the simulation stable and makes "realistic but proportionate to the screen" a straightforward scaling problem rather than a physics problem.

TBD — *Exact platform width and the PX_PER_METER value aren't set — pick values once you know the target canvas size, then keep them in the shared constants file (Section 11).*

4.3 The "freeze on placement" rule

This is the mechanic that lets two pieces lean on each other (like /\) without one sliding out before the other is in place:

- The most recently placed piece is weightless but still collidable — implement as a static (or kinematic) body so it does not fall, but other bodies can still rest against it and be supported by it.

- Placing piece 2 "locks in" piece 1: piece 1 switches from static to a normal dynamic body (gravity applies), while piece 2 becomes the new frozen one. Placing piece 3 then locks piece 2 and freezes piece 3, and so on.
- The frozen piece cannot be pushed by other pieces while it's frozen, since a static body is immovable — this is exactly what makes a leaning pair stable during placement.

4.4 Picking a piece back up

A placed, already-dynamic piece can be picked back up. On pickup, remove it from active collision (e.g., set it as a sensor or temporarily remove the body) so it can be freely repositioned; anything that was leaning on it is no longer supported and physics resolves that naturally — it may fall. When the piece is placed again, it re-enters the freeze rule in 4.3 as if it were brand new.

4.5 Build-phase forgiveness

A visible collapse during the 2-minute build timer does not end the run or penalize the player — they keep building with whatever time and pieces remain. Only the structure's state at the moment the timer hits zero is scored.

4.6 Transition into the Testing Phase

When the build timer ends, clear the freeze state entirely — including on the very last piece placed — so the full structure becomes dynamic at once. Recommend a brief settling window (no disaster force applied yet) before measuring the "official" post-build height, so natural micro-settling isn't mistaken for a collapse.

TBD — *Exact settling duration isn't set. Start with 1.5 seconds and tune from playtesting.*

4.7 Defining "collapsed"

Proposed first-pass rule: track each piece's peak height during the build phase. A piece counts as fallen once its lowest point touches the ground after having been elevated more than roughly one piece-height above it. This distinguishes base pieces that were always near the ground from pieces that genuinely toppled from height.

TBD — *This definition needs playtesting and refinement — flagged explicitly by the original design notes as something to iron out.*

4.8 Despawning stray pieces

Give the platform a defined "tower zone" (a bounding box or radius). Any piece that drifts outside it and stays there past a short grace period is removed from the simulation and excluded from scoring — this implements the rule that an unplaced or wandered-off piece just disappears.

4.9 Material physical properties in-engine

- Friction varies by material and is the only surface property that differs — rougher materials grip better when leaned together.
- Restitution (bounciness) is fixed at 0 for every material.
- Density/mass is derived from each shape's defined mass and size so Matter.js computes correct inertia — this is what makes top-heavy or lopsided stacks realistically prone to tipping.
- Items generated through the real-world-object pipeline (Section 3.4) plug into this same system: Gemini's estimated archetype/footprint/density/friction values are converted into the same density

+ simple-polygon body that every basic shape uses — no special-case physics handling, regardless of whether a piece is a square or an elephant.

4.10 Disaster forces (Testing Phase)

Each disaster is a different force model applied to all dynamic bodies, escalating the longer the tower survives:

Disaster	Force Model	Escalates With
High wind	Continuous or gusting horizontal force, scaled by each piece's exposed area/height.	Wind speed increases over survival time.
Earthquake	Sinusoidal horizontal displacement/acceleration applied to the ground or all bodies.	Amplitude and/or frequency increase over time.
Flooding	A rising water line; submerged pieces get upward buoyant force plus lateral drag, weakening the base.	Water level rises steadily.
Meteor	A single large impulse at a random horizontal position above the tower.	Impact size/force increases with how long the tower has survived.

All four scale in intensity with elapsed survival time, so stability score is a direct measure of how punishing a test the tower withstood.

***TBD** — Exact escalation curves and force magnitudes are placeholders. Start with a simple linear ramp (intensity = base + k × seconds_survived) and tune from playtesting.*

***TBD** — Whether a disaster targets a specific part of the tower (wind hitting the top, flooding hitting the base) is left open — the current force models above do this somewhat implicitly, but it hasn't been decided as an explicit design goal.*

4.11 Test duration

***TBD** — No hard cap is set yet. A 60–90 second cap is suggested so a run resolves quickly and matches daily-puzzle pacing; the stability score (Section 5.2) should normalize against whatever cap is chosen.*

5. Scoring System

5.1 Height score

Measured once, at the moment the build timer ends (the recorded height doesn't change even if the tower later collapses in testing). Height is banded into five tiers at 20% intervals of that day's maximum possible height (the sum of all of that day's pieces, drawn from whatever the current materials pool is, stacked end to end):

Band	% of Max Possible Height
Grounded	0 – 20%
Mound	20 – 40%

Band	% of Max Possible Height
Plateau	40 – 60%
Alps	60 – 80%
Limitless	80 – 100%

Because the day's materials are randomized, "maximum possible height" is recalculated each day from that day's specific set.

5.2 Stability score

A 0.0–5.0 scale, driven by how long the tower survives the Testing Phase relative to the maximum test duration (Section 4.11). A straightforward starting formula: stability = 5 × min(time_survived / max_test_duration, 1).

TBD — *Whether the curve should be linear, stepped, or weighted by how intense a disaster the tower reached (rather than just raw time) is still open.*

5.3 Combined score

Height and stability are weighted 50/50 into one combined score shown to the player — not two separate ratings.

TBD — *The exact combination formula (e.g., normalize both to a 0–100 scale and average) isn't finalized; a plain average of the two normalized scores is a reasonable default to implement first.*

5.4 Leaderboard

LOCKED — One global leaderboard per day, backed by Supabase (Section 7). Each player sees their percentile — "you beat X% of today's players" — in the style of Krillion, rather than just a raw rank. Players are identified by an anonymous device ID with no sign-up step, and can optionally set a display name that appears on the leaderboard instead of an anonymous label — see Section 7 for the full identity model and schema.

6. Daily Challenge Mechanics

- A single global seed per day determines that day's 10 materials (drawn from the current library — basic shapes for now, growing toward real-world objects per Section 3.4) and that day's disaster (drawn from the bank: high wind, earthquake, flooding, meteor) — identical for every player, Wordle-style.
- The seed is a pure function of the date, indexing into whatever the materials library contains at that time — it never triggers a live SerpAPI/Gemini call itself. Growing the library (Section 3.4) is a separate, offline process from picking that day's set.
- One attempt per device per day (Section 7).
- Leaderboard and the day's puzzle reset at a fixed daily rollover.

TBD — *The exact rollover time (midnight UTC vs. local time) isn't decided.*

7. Accounts & Data

LOCKED — Backend: Supabase (Postgres + a thin REST/client-SDK layer). No full account/auth system — identity is a client-generated anonymous device UUID, created on first visit and stored in localStorage, synced to a players row. This removes all sign-in friction for judges and casual players while still letting the one-play-per-day gate and leaderboard work correctly.

LOCKED — Players can optionally set a display name at any time (prompted once, e.g. at onboarding or right before their first results screen, and editable afterward). The name is stored alongside their device UUID and shown on the leaderboard in place of an anonymous placeholder label when set — this gives the percentile/leaderboard hook its "bragging rights" payoff without requiring real accounts.

Schema sketch

- players — id (uuid, primary key, equals the device ID), display_name (text, nullable), created_at
- scores — id, player_id (fk → players.id), date, height_score, stability_score, combined_score, created_at; unique constraint on (player_id, date) to enforce one attempt per day
- daily_puzzle — date (primary key), seed, item_ids (jsonb, referencing the materials library in Section 3.4), disaster_type

TBD — *No protection against a player clearing localStorage / spoofing a new device UUID to replay the same day. Acceptable risk for a hackathon demo; flag as a known limitation rather than something to harden now.*

TBD — *Exact UI moment for the display-name prompt (onboarding vs. first results screen) isn't decided — either works; pick based on what feels least intrusive once the onboarding flow is built.*

8. UI/UX

LOCKED — Visual direction: Minimalist daily-puzzle aesthetic — black/white with a single accent blue, heavy whitespace, clean geometric type (Space Grotesk). Closest reference point is Wordle/Krillion's own look: reads as a polished, confident indie puzzle rather than a game-jam project. Because the visual ceiling is intentionally low, juice (animation, sound, screen shake, particles on collapse/disaster) is what has to carry "Most Entertaining" — see Section 1.2.

LOCKED — Five wireframe screens already exist in Figma, built on a shared component system (buttons, badges, tray pieces, step cards, score cards, leaderboard rows, height ruler) so a single token change cascades everywhere: Daily Intro, Tutorial, Build Phase, Testing Phase, and Results. File: figma.com/design/as3lihY6g66gW3VCrd5sIK/Elevation.

- Tutorial / how-to-play — controls and goal, shown to first-time players (skippable).
- Build Phase HUD — timer, remaining pieces in the tray, live height readout.
- Testing Phase HUD — which disaster is active, survival timer.
- Results screen — height band, stability score, combined score, and leaderboard percentile (with display name if set).

TBD — *Full results-screen content (exact layout of the share/percentile block) still needs a final pass once the scoring formula (Section 5.3) is locked.*

TBD — *Tutorial format — forced on first play vs. always-skippable-with-a-reminder — isn't finalized.*

9. Tech Stack & Hackathon Tracks

9.1 Confirmed stack

- Frontend — Next.js, deployed on Vercel for a fast, shareable public URL.
- Physics — Matter.js (Section 4.1).
- Backend/data — Supabase (Postgres), accessed via the Supabase client SDK; no Supabase Auth product in use — identity is the custom anonymous-device-ID model in Section 7.
- Sponsor-track APIs — SerpAPI (image search) + Gemini (structured JSON reasoning over an object name + image), run offline via a one-off generation script (Section 3.4), never called by the deployed game itself. Keys live in local/CI environment variables and are never sent to the client or committed to the repo.

9.2 Sponsor-track status

The SerpAPI + Gemini integration is a real, working pipeline that produces actual gameplay content (the real-world-object materials library, Section 3.4), not just a demo stub — but it runs as an offline content tool the team runs themselves, not as a feature inside the deployed app. That keeps the deployed game simple and fast while still being a genuine use of both APIs.

TBD — *Worth a quick confirmation with hackathon organizers that an offline/content-generation use of SerpAPI and Gemini (rather than a live in-app call) satisfies that sponsor track's requirements.*

9.3 Target tracks

- Cursor + Grok Bot — build using Cursor for agentic coding and Grok Bot for workflow orchestration.
- Best Novice — auto-qualified by team composition.
- Best Pitch — lead with the daily-puzzle hook and percentile leaderboard.
- Most Entertaining — primary design target; prioritize feel and spectacle over feature count.

10. Development Roadmap

Phases are ordered by dependency, not by calendar time, since the exact hackathon schedule isn't fixed yet — timebox each phase to however many hours the team actually has.

10.1 Team & work split (4 members)

LOCKED — For the MVP build right now: single computer, one driver, mob/pair-programming style — not split into four parallel branches. Phases 0–3 below are tightly interlocking foundational work (repo scaffold, canvas, core physics, the build→test transition); with nobody yet holding context on the codebase, four people on four branches would spend more time merging and re-explaining than four people would save by working in parallel. The other three can still be productive in this window without touching the shared codebase: one preps the Figma-screens-to-component mapping, one researches Matter.js specifics (body types, collision events) ahead of when they're needed, one starts the offline items-library generation script (Section 3.4) independently, since it doesn't touch the game repo at all.

LOCKED — Once the core loop works end to end (roughly: Phase 3 complete — build, place, freeze-rule, and a measured height all working), switch to four parallel tracks so the rest of the roadmap actually

benefits from having four people. Each track below maps to a natural ownership boundary with minimal file overlap, which is what actually prevents merge conflicts — not the org chart.

Track	Owns	Roadmap phases
A — Physics & Gameplay	Collapse detection, disaster force models, freeze-rule edge cases, physics constants tuning	Phase 2 (depth), Phase 4
B — Frontend & UI	Translating the 5 Figma screens into real components, HUD wiring, tutorial, results screen, juice/animation for Most Entertaining	Phase 1, Phase 3 (UI half), Phase 7
C — Backend & Data	Supabase schema, daily seed generation, leaderboard/percentile queries, anonymous device ID + display name flow	Phase 5
D — Materials & Pitch	Offline SerpAPI + Gemini items-library pipeline, item balance tuning, pitch deck (owns the strongest sponsor-track story)	Phase 6, pitch prep

Two integration checkpoints matter more than the track boundaries themselves: Phase 3's build→test transition needs Track A and Track B to agree on exactly what data crosses from the physics sim to the UI (height, per-piece fallen/standing state); and Phase 5's leaderboard needs Track C and Track B to agree on the results-screen data shape before either side builds against a guess. Put these on the calendar as explicit sync points, not something that just happens.

***TBD** — Exact timing of the handoff from single-computer to four-track work isn't fixed to a clock time — it's whenever Phase 3 is genuinely done, which depends on how the hackathon schedule actually unfolds.*

10.2 Phases

- **Phase 0 — Setup: Repo scaffold, Next.js app shell, Matter.js canvas rendering a static platform, CONTEXT.md created (Section 11). Single computer.**
- **Phase 1 — Static placement: Drag shapes from the tray onto the canvas, rotate with Q/E, no gravity yet. Validates that the core interaction feels right before physics is added. Single computer.**
- **Phase 2 — Core physics: Matter.js bodies per shape, the freeze-on-placement rule (4.3), friction-only materials, pickup/reposition (4.4). Single computer.**
- **Phase 3 — Build → Test transition: 2-minute build timer, structure lock, settling window, height measurement and banding (5.1). Single computer through this phase; split into tracks once it's done.**
- **Phase 4 — Disasters (Track A): Implement one disaster end-to-end first — wind is the simplest force model — including escalation and stability scoring, then extend to the other three if time allows.**
- **Phase 5 — Daily system & leaderboard (Track C): Supabase schema (players/scores/daily_puzzle), anonymous device ID + optional display name, global seed → day's materials + disaster, percentile leaderboard query.**
- **Phase 6 — Materials library (Track D): Run the offline SerpAPI + Gemini generation pipeline to build out the real-world-object library, starting from the 3 prototype shapes and growing from**

there (Section 3.4). This is the one phase that scales smoothly with however much time is left — a small library still ships a real, working sponsor-track integration.

- **Phase 7 — Polish & pitch (Track B + whole team): Tutorial screen, results screen, juice (sound, screen shake, particles) aimed at the Most Entertaining track, and the pitch deck.**

If time runs short, cut in this order: materials-library size (Phase 6 — a 5-item library still works, it just demos smaller) → disaster variety beyond the first one (Phase 4) → breaking/durability (already stretch, Section 3.5). Don't cut the core stacking feel or the daily/leaderboard hook — those are what make this feel like a real product rather than a tech demo.

11. context.md — Shared Project Context File

Keep a single living file at the repo root so any teammate — or any AI coding tool (Claude, Cursor, Grok) — can pick the project up cold without being re-briefed from scratch.

11.1 Recommended contents

- Project one-liner and current phase (from the roadmap in Section 10).
- Architecture decisions, now locked: Next.js + Vercel, Matter.js, Supabase (Postgres, no Auth product), anonymous device ID + optional display name, SerpAPI + Gemini as an offline materials-library generation script (not an in-app call).
- Data model definitions — the material/item schema (shared by basic shapes and generated real-world objects alike), the Supabase table schema (Section 7), and the score schema.
- Physics constants currently in use — PX_PER_METER, per-material friction values, disaster force formulas.
- A file/folder structure map.
- How to run the project locally, including which environment variables are required (Supabase URL/key, SerpAPI key, Gemini key) and where to get them — never the key values themselves.
- Open questions — keep this in sync with Section 12 below.
- A running session log — what changed, by whom, and when, so parallel work doesn't collide.

11.2 Process

- Update it at the end of every work session, or whenever a core decision gets finalized.
- Paste its full contents as the first message into any new AI session before asking it to write code.
- Never put API keys or secrets directly in it — reference where they live (e.g., .env.local / Vercel project settings) instead.

12. Open Questions

Everything flagged TBD throughout this document, gathered here for quick reference in a team sync. Resolved items (backend provider, identity model, deployment target, visual direction, team work-split model) have been removed from this list.

- Physics — exact "collapsed" detection rule (4.7); settling-window duration (4.6); platform width and PX_PER_METER scale (4.2); disaster escalation formulas and whether disasters target specific zones of the tower (4.10); max test duration (4.11); rotation degree increment (2.2).
- Materials — numeric mass/size/friction values for the three prototype shapes (3.1); the breaking/health system and any structural-reinforcement bonus (3.5).
- Materials library — initial size of the generated real-world-object library (3.4); whether it lives as a static file or a Supabase table (3.4); source/rotation of the candidate object-name list (3.4).
- Scoring — the stability-score curve (5.2); the exact height + stability combination formula (5.3).
- Daily system — daily rollover time (6).
- Accounts — exact UI moment for the display-name prompt (7); no protection against device-ID spoofing to replay a day, flagged as an accepted hackathon-scope limitation (7).
- UI/UX — full results-screen content (8); tutorial format, forced vs. skippable (8).
- Mobile — touch control scheme, especially for rotation (2.2).
- Team — exact moment to switch from single-computer to the 4-track split isn't tied to a clock time, only to Phase 3 completion (10.1).
- Hackathon logistics — confirm with organizers that an offline (not in-app) use of SerpAPI and Gemini satisfies that sponsor track's requirements (9.2).